Flam assignment: building a clean Indic training corpus

Deadline: 24 hours from when you receive this. Expected effort: 3–4 hours. Work whenever suits you within the 24 hours. Aim to stop after about 4 hours of work and send what you have, with a note on what you'd do next. A reasonable version of every part beats a perfect version of one.

Follow-up. In the 45-minute follow-up we will give you a small file of messy data, run your pipeline on it together, and ask you to change your code live. Make sure you understand everything you submit.

The situation

Falcon, our LLM, is only as good as the data it is trained on, and a large part of our edge is Indian-language quality. Your job is to collect a small Hindi and Hinglish corpus from public sources, turn it into training-ready data, and build an eval set that has provably not leaked into it.

1. Collect (about 45 minutes)

- Gather **at least 10,000 documents** from **at least two** public sources.
- Most of it should be **Hindi in Devanagari**, plus **at least 1,000 documents of Hinglish** (Hindi written in Latin script). If you can't find a usable Hinglish source, tell us what you tried.
- Use only data you are allowed to use. For every source, record the name, URL, license and download date.
- Keep the raw data exactly as you downloaded it. Your pipeline starts from raw.

2. Clean and curate (about 2 hours)

Write a pipeline that runs as:

```
./run.sh <raw_dir> <out_dir>
```

It must:

1. **Read** the raw input in a streaming way, survive malformed records, and count what it skips.
2. **Normalize** text so that things that are the same compare as the same. Think about Devanagari, not just ASCII: Unicode forms, invisible characters, whitespace.
3. **Label language** with exactly one of:
   - `hi` : Hindi in Devanagari, including Devanagari text with English words mixed in
   - `hi-Latn` : Hindi written in Latin script, including code-mixed Hinglish
   - `en` or `other`

   Don't trust labels that came with the source.

4. **Filter** low-quality text: spam, boilerplate, markup, garbled encoding. Be deliberate about what your filters do to text that is legitimate but unusual, such as short proverbs, poems, or tables of numbers.
5. **Deduplicate**: exact and near-duplicate. Choose the threshold by looking at your data, not from a blog post.
6. **Redact PII**: Indian phone numbers, email addresses, Aadhaar numbers and PAN numbers. Replace them with `<PHONE>`, `<EMAIL>`, `<AADHAAR>` and `<PAN>`. Numbers that are not PII must survive.
7. **Keep a ledger**: one row per input document saying whether it was kept or dropped, at which stage, why, and which document it duplicates (if any).

3. Eval set and contamination (about 45 minutes)

- Build an **eval set of about 200 items** from data you hold out from training. Say in one line what it is meant to measure.
- Check your training corpus for **leaks of eval items**: exact copies, copies that differ only after normalization, and near-copies. Consider whether a Devanagari eval item could appear in training written in Latin script.
- Report what you find, and remove the leaking training documents.

4. Write-up: DECISIONS.md (about 30 minutes)

One page. Use numbers from your own run, not generalities. Bullet points are fine.

1. **Sources:** name, license, document count and size for each.
2. **Funnel:** how many documents went in and came out at each stage, per language.
3. **Near-duplicates:** your method and threshold, and how you chose them. Show three pairs close to the threshold with your call on each.
4. **Filters:** three documents your filters removed that they shouldn't have, or your argument that there are none.
5. **Contamination:** your method, how many eval items were affected, and one candidate match you rejected and why.
6. **Reproducibility:** how someone rebuilds exactly this dataset six months from now.
7. **Scale:** if this were 2 TB on object storage, what breaks first in your design? One short paragraph.
8. **Wrap-up:**
    - what you didn't get to, in priority order
    - how long you actually spent

Requirements

- **Python.** Put downloading in a separate `download.sh`; `run.sh` must work offline.
- **Streaming:** process the input as a stream, as if the full corpus would not fit in memory.
- **Deterministic and safe to rerun:** the same input gives the same output. If the run is killed partway and restarted, the final output must match an uninterrupted run, with no duplicated rows.

- **No hardcoding:** your pipeline must work on data it has never seen, including the file we bring to the follow-up.

Output

Path	Contents
`<out_dir>/clean/`	`*.parquet` or `*.jsonl.gz`, one row per kept document: `id`, `text`, `lang`, `source`
`<out_dir>/ledger.*`	One row per input document: `id`, `status` (`kept` or `dropped`), `stage`, `reason`, `dup_of`
`<out_dir>/eval.jsonl`	Your eval set
`<out_dir>/contamination.jsonl`	One row per affected eval item: `eval_id`, `train_ids`, `method`
`<out_dir>/manifest.json`	Sources, input file hashes, your thresholds, and counts per stage

What to send

A git repo, zipped or as a link, containing `download.sh`, `run.sh`, your source, `DECISIONS.md`, your eval set, and a **1,000-row random sample** of your cleaned output. Don't send the full dataset. Keep the commit history; we read it.

How we evaluate

- **Data quality:** we read samples of your output and your ledger.
- **Judgment:** whether the numbers in DECISIONS.md are real and the trade-offs are reasoned.
- **Engineering:** streaming, failure handling, determinism, and code someone else can read.
- **Follow-up:** your pipeline on our file, changing it live, and how a data decision shows up in model behaviour.

We do not evaluate UIs, dashboards, notebooks or polish. Where this brief is ambiguous, make a call, write it down in DECISIONS.md and move on.